```
...
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Book post(@Valid @RequestBody Book book) {
  return bookService.addBookToCatalog(book);
}
@PutMapping("{isbn}")
public Book put(@PathVariable String isbn, @Valid @RequestBody Book book) {
  return bookService.editBookDetails(isbn, book);
}
...
```

Spring lets you handle error messages in different ways. When building an API, it's good to consider which types of errors it can throw, since they are just as important as the domain data. When it's a REST API, you want to ensure that the HTTP response uses a status code that best fits the purpose and includes a meaningful message to help the client identify the problem.

When the validation constraints we have just defined are not met, a `MethodArgumentNotValidException` is thrown. What if we try to fetch a book that doesn't exist? The business logic we previously implemented throws dedicated exceptions (`BookAlreadyExistsException` and `BookNotFoundException`). All those exceptions should be handled in the REST API context to return the error codes defined in the original specification.

To handle errors for a REST API, we can use the standard Java exceptions and rely on a `@RestControllerAdvice` class to define what to do when a given exception is thrown. It's a centralized approach that allows us to decouple the exception handling from the code throwing the exception. In the `com.polarbookshop.catalogservice .web` package, create a `BookControllerAdvice` class as follows.

Listing 3.14 The advice class defining how to handle exceptions